SG DMA — 검증 요구사항 (SOR)

항목	내용
문서 ID	SGDMA-SOR-rel-v2.0.0
날짜	2026-10-16
이전 판	SGDMA-SOR-rel-v1.0.0 (요구사항 61건)
이번 판	**테스트 15개** (원래 61건 중 58건을 15개에 묶고, 3건은 제외)
만든 방법	작성 Agent 5개가 각자 15개안을 만들고 리뷰 Agent 5개가 승인할 때까지 검사 → 5개안을 비교해 최종 선택 (`SGDMA-SOR-INTEGRATION-v1.0`)
검증 대상	`rtl/sg_dma/sgdma_top.v` — ADI `axi_dmac` 기반. 메모리→메모리, SG 켬, 클럭 1개, 32비트 주소
함께 보는 문서	`SGDMA-SFR-rel-v2.0.0` (레지스터 설명), `SGDMA-SANITY-rel-v2.0.0` (빠른 회귀 검사)

목차

1. SG DMA 처음 보는 분께
2. 이 문서를 읽는 법
3. 테스트 15개 한눈에 보기
4. 테스트 15개 상세
5. 제외한 요구사항
6. 원래 61건 대응표
7. 개정 이력

1. SG DMA 처음 보는 분께

1.1 비유로 이해하기

DMA는 CPU 대신 메모리 내용을 옮겨 주는 "심부름꾼" 회로입니다.

보통 DMA는 심부름을 하나 끝낼 때마다 CPU에게 돌아와 다음 일을 받아 갑니다.

SG(Scatter-Gather) DMA는 다릅니다. CPU가 메모리에 **심부름 목록**을 적어 두고, 목록의 첫 장이 어디 있는지만 알려 줍니다. SG DMA는 목록을 한 장씩 읽으면서 끝까지 처리하고, 다 끝나면 CPU에게 **한 번만** 알려 줍니다.

목록의 한 장 한 장을 **descriptor**(전송 1건의 할 일 메모)라고 부릅니다. 메모마다 "어디서 읽고, 어디에 쓰고, 몇 바이트이고, 다음 메모는 어디 있다"가 적혀 있습니다. 메모들이 줄줄이 이어진 것을 **체인**이라고 합니다.

- **gather(모으기)**: 여기저기 흩어진 데이터를 읽어 한 곳에 이어 붙여 씁니다.
- **scatter(나눠 쓰기)**: 한 곳의 데이터를 읽어 여러 곳에 나눠 씁니다.

1.2 전송 한 번은 이렇게 합니다

이 순서를 "**표준 순서**"라고 부르고, 아래 테스트에서 계속 씁니다.

1. 메모리에 descriptor 체인을 적는다. 마지막 장만 `flags = 0x3`(마지막 장 + 끝나면 알림).
2. `IRQ_MASK(0x080) ← 0x1` — "완료" 알림만 받겠다고 설정.
3. `CONTROL(0x400) ← 0x5` — 켜기(ENABLE) + SG 모드(HWDESC).
4. `SG_ADDRESS(0x47C) ← 첫 descriptor 주소`.
5. `TRANSFER_SUBMIT(0x408) ← 0x1` — 출발.
6. 완료를 기다린다 (`irq` 신호 또는 `TRANSFER_DONE` 반복 읽기).
7. `IRQ_PENDING(0x084)`을 읽고, 읽은 값을 그대로 다시 써서 알림을 지운다.

> ⚠️ **3번과 5번 순서를 바꾸면 아무 일도 일어나지 않고, 에러도 나지 않습니다.** 켜기 전에 출발 버튼을 누르면 조용히 무시됩니다. 신입이 가장 많이 걸리는 함정입니다.

1.3 꼭 알아야 할 레지스터 7개

주소	이름	하는 일	주의할 점
`0x400`	`CONTROL`	비트0 켜기(ENABLE), 비트1 일시정지(PAUSE), 비트2 SG 모드(HWDESC)	처음엔 0. SG를 쓰려면 `0x5`를 써야 함. 켜기를 0으로 내리면 진행 상태가 지워짐
`0x47C`	`SG_ADDRESS`	첫 descriptor 주소	아래 3비트는 버려짐 (8바이트 단위로 맞춰짐)
`0x408`	`TRANSFER_SUBMIT`	1을 쓰면 출발 요청. 받아들여지면 **저절로 0**이 됨	켜기 전에 쓰면 조용히 무시됨. 1인 동안 또 1을 써도 요청이 늘지 않음
`0x080`	`IRQ_MASK`	1이면 그 알림을 **막음**	**처음 값이 `0x3`** — 처음엔 알림이 전부 막혀 있음
`0x084`	`IRQ_PENDING`	지금 올라와 있는 알림. 1을 쓰면 지워짐	**실제로 지워지는 곳은 `IRQ_SOURCE`**
`0x088`	`IRQ_SOURCE`	막혀 있든 아니든 일어난 일을 모두 기록	막아 둔 동안에도 쌓임. 막기를 풀면 쌓인 알림이 바로 올라옴
`0x428`	`TRANSFER_DONE`	전송 번호(0~3)별 완료 표시	SG에서는 **체인 하나가 전송 1건**

1.4 descriptor 한 장의 모양 (48바이트)

위치	내용
`+0x00`	`flags` — 비트0 LAST(마지막 장), 비트1 EOT_IRQ(이 장이 끝나면 알림)
`+0x04`	`id` — 이 장의 이름표. `DESCRIPTOR_ID(0x454)`에서 보임
`+0x08`	목적지 주소 (아래 3비트 무시)
`+0x10`	소스 주소 (아래 3비트 무시)
`+0x18`	다음 장 주소 (LAST=0일 때만 씀)
`+0x24`	`x_len` = **옮길 바이트 수 − 1** (아래 24비트만 씀)
`+0x20`, `+0x28`, `+0x2C`	2D 전송용. 이 구성에서는 무시. 0으로 채움

> ⚠️ `x_len`은 "바이트 수 − 1"입니다. `x_len = 0`이면 **1바이트**, `x_len = 0xFFFFFF`이면 **16 MiB**(가장 긴 전송)입니다. 그래서 **길이 0인 descriptor는 이 형식으로 만들 수 없습니다.** (v1.0.0의 "길이 0 = `0xFFFFFF`"라는 표현은 틀린 것이어서 고쳤습니다.)

DMA는 descriptor 한 장을 **한 번에 48바이트**(8바이트 × 6번) 읽습니다.

1.5 이 구성만의 특징

소스와 목적지가 **둘 다 메모리**라서, **1바이트 단위로 아무 길이나** 보낼 수 있습니다 (`DMA_LENGTH_ALIGN = 0`). ADI가 만든 공식 testbench는 이 조합을 검증하지 않습니다. 그래서 이 부분(T-07)이 우리 검증에서 특히 중요합니다.

1.6 처음 보는 말 풀이

말	쉬운 뜻
DUT	검증하는 회로. 여기서는 `sgdma_top`
RTL	회로를 글로 적은 설계 코드 (Verilog)
TB (testbench)	DUT에 신호를 넣고 결과를 검사하는 시뮬레이션 환경
Sanity (SAN-xxx)	RTL을 고칠 때마다 돌리는 빠른 검사 46개 (`SGDMA-SANITY`)
AXI	칩 안에서 데이터를 주고받는 표준 버스 규칙
AR / R	읽기 주소를 보내는 통로 / 읽은 데이터가 오는 통로
AW / W / B	쓰기 주소 통로 / 쓰기 데이터 통로 / 쓰기 결과 응답 통로
beat	버스의 한 박자. 이 DUT는 한 박자에 8바이트
burst	주소 한 번으로 여러 박자를 묶어 보내는 데이터 덩어리. 이 DUT는 최대 128바이트(16박자)
`ARLEN`	burst의 박자 수 − 1
`WSTRB`	쓰기 박자에서 "진짜로 쓸 바이트"를 표시하는 신호
`WLAST` / `RLAST`	burst의 마지막 쓰기 박자 / 마지막 읽기 박자 표시
OKAY / SLVERR / DECERR	버스 응답: 성공 / 실패 / "그런 주소 없음"
`AxCACHE` / `AxPROT`	버스 요청에 붙는 캐시 속성 / 보호 속성
W1C	1을 쓰면 지워지는 비트
SOT / EOT	전송 접수(Start) / 전송 완료(End). `IRQ_SOURCE`의 비트0 / 비트1. descriptor의 EOT_IRQ 플래그는 "이 장이 끝나면 완료 알림을 내라"는 뜻
prefetch	다음 할 일 메모를 미리 읽어 두기
backpressure	받는 쪽이 천천히 받아서 보내는 쪽이 기다려야 하는 상황
idle	버스에 아무 거래도 없는 쉬는 상태
hang	멈춰서 끝나지 않는 상태 (이 문서의 정확한 뜻은 2.3절)
assertion	규칙을 어기면 자동으로 잡아내는 검사 코드
guard 영역	목적지 끝 바로 뒤 16바이트. 써지면 안 되는 감시 구역

2. 이 문서를 읽는 법

2.1 판정의 네 종류

표시	뜻
판정	기대값과 다르면 FAIL
참고 체크	기대값과 달라도 FAIL이 아님. "구현이 바뀌었다"는 신호로 기록 (ADI가 약속한 동작이 아님)

표시	뜻
관찰 기록	비교 없이 값만 남김. 동작을 정하거나 디버깅할 때 씀
관측 후 확정	아직 정답이 정해지지 않은 동작. 첫 실행에서 관측해 팀이 정답을 정한 뒤 "판정"으로 바꿈

두 가지 규칙:

- `DESCRIPTOR_ID`의 전송이 **끝난 뒤** 값은 항상 **참고 체크**입니다.
- 디버그 레지스터(`0x434` ~ `0x444`, 예: `DBG_STATUS`)는 항상 **관찰 기록**입니다. 합격 기준에 쓰지 않습니다.

2.2 모든 테스트에 공통으로 보는 것 (C1 ~ C6)

#	조건
C1	시간 초과 없음 (기본 50,000 클럭)
C2	목적지 데이터가 소스와 바이트 하나하나까지 같음
C3	guard 영역(목적지 끝 바로 뒤 16바이트)이 한 번도 써지지 않음
C4	완료 알림이 왔음 — 표준 순서 7번에서 **지우기 전에 읽은** `IRQ_PENDING[1]`이 1. (알림을 막아 둔 시험에서는 `IRQ_SOURCE[1]` 또는 `TRANSFER_DONE`의 해당 번호 비트가 1)
C5	AXI 규칙 위반 0건 (보강 B1 필요)
C6	시뮬레이션 에러 메시지(`$error`) 0건

> ⚠️ C4는 **알림을 지우기 전에** 확인합니다. `IRQ_PENDING`에 쓰면 `IRQ_SOURCE`도 지워지므로, 지운 뒤에 보면 정상인데도 FAIL이 납니다.

각 테스트의 "통과 기준"에는 이 공통 조건 **말고 추가로** 볼 것만 적었습니다. 공통 조건을 적용하지 않는 곳은 각 테스트의 "예외" 줄에 적었습니다.

2.3 "hang 없음"의 뜻

어떤 전송은 **원래 스스로 끝나지 않습니다** (예: 자기 자신을 가리키는 체인, 16 MiB 전송). 그래서 이 문서에서 "hang 없음"은 아래 뜻입니다.

1. 시간 안에 스스로 끝나면 hang이 아닙니다.
2. 끝나지 않더라도, `CONTROL ← 0x4`(켜기만 끄기)를 쓴 뒤
   - (가) 버스가 50,000 클럭 안에 idle로 돌아오고,
   - (나) `CONTROL ← 0x5`로 다시 켜서 T-03 전송을 보내면 정상으로 끝나면,

   hang이 아닙니다.
3. (가)나 (나)가 실패하면 **hang**입니다.

2.4 테스트 시작 전 공통 준비

모든 테스트는 시작 전에 아래를 합니다. 단, T-01과 T-09 1단계("리셋 직후 읽기")는 **HW 리셋을 새로 건 직후**, 아무것도 쓰기 전에 합니다. 여러 테스트를 한 시뮬레이션에서 이어 돌린다면 이 두 단계 앞에서 리셋을 다시 겁니다.

1. `CONTROL ← 0x0` (끄기)
2. `IRQ_PENDING ← 0x3` (남은 알림 지우기)

3. 목적지 메모리와 "써졌음" 기록 지우기 (보강 B5)

2.5 공통 메모리 영역

영역	주소	용도
DESC	`0x0000_1000` ~ `0x0000_1FFF`	descriptor 체인
SRC	`0x0001_0000` ~ `0x0001_FFFF`	소스 데이터
DST	`0x0002_0000` ~ `0x0002_FFFF`	목적지 데이터
BOUND	`0x0000_0F00` ~ `0x0000_10FF`	4 KB 경계 시험 전용 (DESC와 일부 겹치므로 그 시험 때만 사용)
ERR	`0x0003_0000` ~ `0x0003_FFFF`	에러 응답을 돌려줄 영역
LONG	`0x0004_0000` ~ `0x0004_FFFF`	긴 체인용 descriptor

이 표에 없는 주소는 TB에서 위 영역 안에 겹치지 않게 정하고 로그에 남깁니다.

2.6 testbench 보강 목록

지금 TB(`tb/sg_dma/tb_sg_dma.sv`)에는 아래 기능이 **없습니다.** 보강하기 전에는 해당 판정을 "보류"로 둡니다. 테스트 본문에 `[B번호]`로 표시했습니다.

#	보강할 기능	쓰는 테스트
B1	AXI 규칙 검사기(assertion): 주소·데이터가 받기 전까지 유지되는지, burst 모양(`ARLEN` 등), `WLAST` 위치, 4 KB 경계, `AxCACHE`/`AxPROT`, `RLAST` 시각 기록	공통 C5, T-03, T-07, T-13, T-14
B2	버스 요청 자동 기록기: 포트별로 AR/AW 주소·박자 수·횟수·시각을 기록하고, 각 테스트의 통과 기준 표에서 만든 기대 목록과 비교 (지금은 화면 출력만 있음)	T-03 ~ T-07, T-10, T-11, T-13
B3	에러 응답 주입: 정한 주소 영역의 읽기/쓰기 응답을 SLVERR 또는 DECERR로 바꿈. 이때 읽기 데이터는 **모두 0**	T-15
B4	느린 응답 주입: 포트별로 `READY` 신호를 무작위로 늦춤	T-14
B5	반복 사이 초기화: 메모리 내용과 "써졌음" 기록을 지움	모든 반복 시험
B6	전송 중 레지스터 반복 읽기: 전송과 동시에 N 클럭마다 레지스터를 읽음	T-08, T-11, T-12

3. 테스트 15개 한눈에 보기

번호	제목	중요도	원래 ID 수	관련 Sanity
T-01	레지스터 버스와 기본 구성 확인	중간	6	SAN-001, 002
T-02	일부러 꺼 둔 비트와 레지스터 접근 규칙	중간	4	SAN-007
T-03	descriptor 1장으로 복사하기	높음	9	SAN-004
T-04	체인 따라가기와 모으기(gather)	높음	3	SAN-005
T-05	나눠 쓰기(scatter)	높음	1	없음
T-06	체인 길이를 1장부터 32장까지	높음	1	SAN-005 (3장만)
T-07	1바이트 단위 길이 전송	높음	5	SAN-004 (256바이트, 참고 실측만)
T-08	중간 알림(EOT)과 descriptor 번호	중간	4	SAN-005, 006
T-09	알림 막기(MASK)와 지우기(W1C)	높음	4	SAN-003, 006

번호	제목	중요도	원래 ID 수	관련 Sanity
T-10	출발 버튼 규칙과 연속 예약	높음	8	SAN-004 (완료 비트만)
T-11	전송 중 끄기(ENABLE=0)와 다시 켜기	높음	2	없음
T-12	일시정지(PAUSE)와 다시 시작	중간	2	없음
T-13	주소 정렬과 4 KB 경계	중간	3	SAN-007 (일부)
T-14	느린 버스(backpressure)와 AXI 속성	중간	2	SAN-001 (레지스터 값만)
T-15	에러 응답과 잘못된 descriptor	높음	4	없음
	합계		**58**	

추천 진행 순서

1. 보강 B1, B2, B5
2. T-01 → T-02 → T-03 → T-04 → T-09 (여기까지는 Sanity를 다시 확인하는 단계)
3. B6 → T-05 → T-06 → T-07 → T-10 → T-08
4. T-11 → T-12 → T-13
5. B4 → T-14, B3 → T-15

> T-01과 T-09 1단계 앞에서는 HW 리셋을 다시 겁니다 (2.4절).

---

4. 테스트 15개 상세

T-01. 레지스터 버스와 기본 구성 확인

- **무엇을 확인하나**: 레지스터를 읽고 쓸 수 있는지, 고정값 레지스터가 우리가 정한 설정과 맞는지 봅니다.
- **왜 중요한가**: 버스가 안 되면 다른 모든 결과를 믿을 수 없습니다. 구성이 다르면 다른 테스트의 기대값이 모두 틀어집니다.
- **어떻게 확인하나**:
  1. 리셋 직후 `0x000`, `0x004`, `0x00C`, `0x010`, `0x014`를 읽는다.
  2. `VERSION`이나 `IDENTIFICATION`이 틀리면 **나머지 테스트를 모두 건너뛴다.**
  3. `SCRATCH(0x008)`에 `0xA5A5_5A5A`를 쓰고 읽는다.
  4. 반대 패턴 `0x5A5A_A5A5`를 쓰고 읽는다. (한 패턴만 쓰면 고장 나서 고정된 비트를 못 찾습니다.)
  5. `0x0000_0000`을 쓰고 읽어 원래대로 돌려놓는다.
- **통과 기준** (예외: 전송이 없으므로 C1~C5 적용 안 함):

레지스터	기대값
`VERSION` (0x000)	`0x0004_0565`
`PERIPHERAL_ID` (0x004)	`0x0000_0000`
`IDENTIFICATION` (0x00C)	`0x444D_4143` ("DMAC")
`INTERFACE_DESCRIPTION_1` (0x010)	`0x3807_0303`
`INTERFACE_DESCRIPTION_2` (0x014)	`0x0000_0000`
`SCRATCH` 3번	쓴 값과 같음

> `INTERFACE_DESCRIPTION_2`를 `0x30`으로 계산한 옛 문서가 있었는데 틀린 값입니다. 기대값은 `0x0000_0000`입니다. `0x004`, `0x010`, `0x014`가 틀리면 버스 고장보다 **기대값이 틀렸을 가능성**을 먼저 봅니다.

- **흡수한 원래 요구사항**: REG-001, REG-002, REG-003, REG-004, REG-005, REG-020
- **중요도**: 중간

T-02. 일부러 꺼 둔 비트와 레지스터 접근 규칙

- **무엇을 확인하나**: 설정으로 꺼 둔 비트가 정말 안 바뀌는지, 읽기 전용 레지스터·없는 주소·일부 바이트만 쓰기가 어떻게 동작하는지 봅니다.
- **왜 중요한가**: 이걸 모르면 "써도 안 바뀐다"는 **정상 동작**을 버그로 보고합니다. 이 IP는 없는 주소에도 에러를 주지 않아서, 읽은 값 비교가 주소 실수를 잡는 유일한 방법입니다.
- **어떻게 확인하나**:
  1. 아래 표의 레지스터에 `0xFFFF_FFFF`를 쓰고 읽는다. `CONTROL`에 쓰면 켜기·일시정지·SG 모드가 모두 1이 되므로, **되돌리기 전까지 `TRANSFER_SUBMIT`을 쓰지 않는다.**
  2. 꺼진 레지스터와 없는 주소(`0x0F0`, `0x600`)를 읽는다.
  3. `VERSION(0x000)`과 `IRQ_SOURCE(0x088)`에 `0xFFFF_FFFF`를 쓰고 읽는다. 쓰기 응답도 기록한다.
  4. `SCRATCH`에 `0xFFFF_FFFF`를 먼저 쓴 뒤, `WSTRB = 4'b0001`(맨 아래 1바이트만)로 `0x0000_00FF`를 쓰고 읽는다.
  5. **반드시 되돌린다**: `CONTROL ← 0x0`, `FLAGS ← 0x2`, `0x410` / `0x414` / `0x47C ← 0x0`, `SCRATCH ← 0x0`.
  6. `0x450`은 **절대 읽지 않는다.** 읽으면 내부 대기열이 하나 빠져서 상태가 망가집니다.
- **통과 기준** (예외: C1~C5 적용 안 함):

동작	기대값	이유
`0x400` 쓰기 후 읽기	`0x0000_0007`	프레임락 기능 꺼짐
`0x40C` 쓰기 후 읽기	`0x0000_0006`	반복(cyclic) 기능 꺼짐
`0x410` / `0x414` / `0x47C` 쓰기 후 읽기	`0xFFFF_FFF8`	8바이트 단위로 맞춰짐
`0x490` / `0x494` / `0x4BC` 쓰기 후 읽기	`0x0000_0000`	32비트 주소 구성
`0x41C`, `0x458` 읽기	`0x0000_0000`	2D·프레임락 꺼짐
`0x420`, `0x424`, `0x45C` 읽기	`0x0000_0000`	꺼진 레지스터 (RTL 분석 값, 첫 측정)
`0x0F0`, `0x600` 읽기	`0x0000_0000`, 에러 응답 없음	없는 주소
`VERSION` 쓰기 후 읽기	`0x0004_0565` 그대로, 응답 OKAY	읽기 전용
`IRQ_SOURCE` 쓰기 후 읽기	쓰기 전 값 그대로, 응답 OKAY	읽기 전용
`SCRATCH` 1바이트 쓰기 후 읽기	**`0x0000_00FF`**	`WSTRB`를 무시하고 32비트 전체를 씀

- **흡수한 원래 요구사항**: REG-019, REG-021, REG-022, REG-023
- **중요도**: 중간

T-03. descriptor 1장으로 복사하기

- **무엇을 확인하나**: descriptor 1장을 읽고 그대로 256바이트를 정확히 복사하는지 봅니다. descriptor를 48바이트 한 번에 읽는지, 각 칸을 맞게 해석하는지, LAST 플래그에서 끝나는지 함께 봅니다.

- **왜 중요한가**: SG의 가장 작은 단위입니다. 여기가 틀리면 체인, gather, scatter가 모두 틀립니다. 길이를 "바이트 수 − 1"로 잘못 읽으면 1바이트씩 넘치거나 모자랍니다.
- **어떻게 확인하나**:
  1. `0x0001_0000`에 256바이트 패턴을 쓴다.
  2. `0x0000_1000`에 descriptor를 쓴다: `flags = 0b11`, `id = 0xA0`, 소스 `0x0001_0000`, 목적지 `0x0002_0000`, 다음 `0x0`, `x_len = 255`.
  3. 표준 순서로 출발한다 (`SG_ADDRESS ← 0x0000_1000`). `SG_ADDRESS`를 다시 읽는다.
  4. 세 포트(descriptor 읽기 `m_sg_axi`, 소스 읽기 `m_src_axi`, 목적지 쓰기 `m_dest_axi`)의 요청을 모두 기록한다 [B2].
  5. 완료 후 알림을 지우고 `TRANSFER_DONE`을 읽는다.
- **통과 기준**:

보는 곳	기대값
`SG_ADDRESS` 읽기	`0x0000_1000`
descriptor 읽기	주소 `0x1000`, `ARLEN = 5`(6박자), 박자 크기 8바이트, 주소 증가 방식(INCR), **딱 1번**
소스 읽기	`0x1_0000`(16박자), `0x1_0080`(16박자) — 128바이트씩 2번
목적지 쓰기	`0x2_0000`(16박자), `0x2_0080`(16박자)
읽은 양 = 쓴 양	256바이트
guard `0x0002_0100` ~ +16바이트	안 써짐
`TRANSFER_DONE`	해당 번호 비트 = 1
`DESCRIPTOR_ID` (참고 체크)	`0xA0`

> descriptor 읽기 모양(`ARLEN = 5`, 8바이트, INCR)은 B1이 생기면 **모든 테스트에서 자동으로 계속 감시**합니다.

- **흡수한 원래 요구사항**: REG-007, REG-015, DESC-001, DESC-002, DESC-003, DESC-007, FUNC-001, FUNC-009, AXI-001
- **중요도**: 높음

T-04. 체인 따라가기와 모으기(gather)

- **무엇을 확인하나**: "다음 장 주소"를 따라 descriptor 3장을 순서대로 읽는지, 흩어진 소스 3곳을 목적지 한 곳에 이어 붙이는지 봅니다. 다음 장을 미리 읽어 두는지(prefetch)도 봅니다.
- **왜 중요한가**: 체인을 따라가지 못하면 SG가 아닙니다. 체인 전체에 알림이 **한 번만** 와야 CPU 일이 줄어듭니다.
- **어떻게 확인하나**:
  1. 아래 표대로 소스 3곳과 descriptor 3장을 쓴다. 마지막 장만 `flags = 0b11`.
  2. 출발 전에 `TRANSFER_ID`를 읽어 둔다.
  3. 표준 순서로 출발한다 (`SG_ADDRESS ← 0x0000_1100`).
  4. 세 포트의 요청 주소와 시각을 기록하고 [B2], `irq`가 몇 번 오는지 센다.
  5. 완료 후 `TRANSFER_ID`를 다시 읽는다. 조각마다 따로 비교해서, 틀리면 어느 조각인지 알 수 있게 한다.

#	descriptor 주소	id	flags	소스	목적지	길이
0	`0x0000_1100`	`0xB0`	`0b00`	`0x0001_1000`	`0x0002_1000`	128

#	descriptor 주소	id	flags	소스	목적지	길이
1	`0x0000_1140`	`0xB1`	`0b00`	`0x0001_3000`	`0x0002_1080`	256
2	`0x0000_1180`	`0xB2`	`0b11`	`0x0001_5000`	`0x0002_1180`	64

- **통과 기준**:

보는 곳	기대값
descriptor 읽기 순서	`0x1100` → `0x1140` → `0x1180`, **딱 3번**
미리 읽기(prefetch)	첫 목적지 쓰기 전에 descriptor 읽기 3번이 모두 나옴. 참고 실측(Sanity): descriptor 읽기 4,215 / 4,295 / 4,375 ns, 첫 소스 읽기 4,335 ns, 첫 목적지 쓰기 4,515 ns
목적지 쓰기 주소	`0x2_1000`, `0x2_1080`, `0x2_1100`, `0x2_1180` — 이어짐. 소스 조각 경계와 상관없이 128바이트씩 다시 나뉨
조각 0 / 1 / 2 데이터	각각 일치
guard `0x0002_11C0` ~ +16바이트	안 써짐
`irq`	**1번만**
`TRANSFER_ID`	출발 전보다 **1만** 늘어남 (0~3을 돌아감) — 체인 하나 = 전송 1건
`DESCRIPTOR_ID` (참고 체크)	`0xB2`

- **흡수한 원래 요구사항**: DESC-004, DESC-009, FUNC-002
- **중요도**: 높음

T-05. 나눠 쓰기(scatter)

- **무엇을 확인하나**: 이어진 소스 448바이트를 서로 떨어진 목적지 3곳에 나눠 쓰는지 봅니다. gather의 반대 방향입니다.
- **왜 중요한가**: 두 방향이 다 되어야 "Scatter-Gather"입니다. 지금 Sanity는 gather만 봤고 **scatter는 아무도 확인하지 않았습니다.** 목적지 사이 빈 곳에 잘못 쓰면 남의 데이터가 깨집니다.
- **어떻게 확인하나**:

  1. `0x0001_8000`부터 448바이트 이어진 패턴을 쓴다. (T-04와 다른 패턴을 쓴다.)
  2. 아래 표대로 descriptor 3장을 쓴다. 마지막 장만 `flags = 0b11`. `id`는 서로 다르게 정한다.
  3. 표준 순서로 출발한다 (`SG_ADDRESS ← 0x0000_1200`).
  4. 소스 읽기와 목적지 쓰기 주소를 기록한다 [B2].
  5. 목적지 3곳과 그 사이 빈 곳을 검사한다.

#	descriptor 주소	소스	길이	목적지
0	`0x0000_1200`	`0x0001_8000`	128	`0x0002_2000`
1	`0x0000_1240`	`0x0001_8080`	256	`0x0002_4000`
2	`0x0000_1280`	`0x0001_8180`	64	`0x0002_6000`

- **통과 기준**:

항목	기대
목적지 3곳	각각 소스의 해당 부분과 일치, 그리고 **이번 전송에서 실제로 써졌음** ("써졌음" 기록으로 확인)
빈 곳 `0x0002_2080` ~ `0x0002_3FFF`, `0x0002_4100` ~ `0x0002_5FFF`	한 바이트도 안 써짐
guard `0x0002_6040` ~ +16바이트	안 써짐
소스 읽기 주소	이어짐
목적지 쓰기 주소	세 곳으로 나뉨
`irq`	1번

- **흡수한 원래 요구사항**: FUNC-003
- **중요도**: 높음

T-06. 체인 길이를 1장부터 32장까지

- **무엇을 확인하나**: descriptor가 1장일 때부터 32장일 때까지 끝까지 처리하는지 봅니다.
- **왜 중요한가**: 체인 길이에 따라 내부 대기열이 차고 비는 모양이 바뀝니다. 어떤 길이에서 체인이 끊기거나 멈추면 실제 드라이버에서 바로 문제가 됩니다. Sanity는 3장만 봤습니다.
- **어떻게 확인하나**:
  1. N = 1, 2, 4, 8, 16, 32로 반복한다. 조각 하나는 64바이트다.
  2. 반복마다 메모리를 지우고 [B5], 소스 N곳에 반복마다 다른 패턴을 쓴다.
  3. descriptor N장을 `0x0000_1000 + (i × 64)`에 쓴다. 마지막 장만 `flags = 0b11`.
  4. 표준 순서로 출발하고, descriptor 읽기 횟수를 세며 [B2] 모든 조각과 guard를 검사한다.
  5. 시간이 되면 N = 128을 LONG 영역에 두고 한 번 더 한다.
- **통과 기준**:

항목	기대
descriptor 읽기 횟수	**딱 N번**
모든 조각	일치
`irq`	길이와 상관없이 **1번**
N = 32 (그리고 128)	시간 초과 없음
`DESCRIPTOR_ID` (참고 체크)	마지막 장의 id

- **흡수한 원래 요구사항**: FUNC-004
- **중요도**: 높음

T-07. 1바이트 단위 길이 전송

- **무엇을 확인하나**: 1, 7, 129바이트처럼 8의 배수가 아닌 길이도 정확히 그만큼만 옮기는지 봅니다. 큰 전송이 128바이트 덩어리로 나뉘는지, 소스 덩어리를 다 받은 뒤에 쓰기를 시작하는지도 봅니다.

- **왜 중요한가**: 이 구성만의 특징이고 **ADI testbench가 검증하지 않는 부분**입니다. 우리가 고친 RTL 한 곳(0-replication 패치: "0번 반복"이라는 Verilog 문법을 시뮬레이터가 받아들이도록 고친 것, SFR §4.4.6 참고)도 이 경로에 있습니다. 1바이트라도 넘치면 옆 데이터가 깨집니다.
- **어떻게 확인하나**:
  1. 소스 `0x0001_0000`, 목적지 `0x0002_0000`으로 고정한다 (주소 정렬 문제와 섞이지 않게).
  2. 길이 1, 2, 7, 8, 9, 63, 64, 65, 127, 128, 129, 255, 256, 512바이트로 descriptor 1장 전송을 반복한다 (`x_len = 길이 − 1`).
  3. **반복마다 목적지와 "써졌음" 기록을 지운다** [B5].
  4. 마지막 쓰기 박자의 `WSTRB`와 `WLAST`를 자동 검사한다 [B1].
  5. 512바이트에서는 소스 덩어리마다 마지막 읽기 데이터(`RLAST`)를 받은 시각과, 목적지 쓰기 주소가 나간 시각을 기록한다 [B1, B2].
- **통과 기준**:

길이 (바이트)	마지막 박자에서 진짜 쓰는 바이트 수
1 / 2 / 7 / 8	1 / 2 / 7 / 8
9	1
63 / 64 / 65	7 / 8 / 1
127 / 128 / 129	7 / 8 / 1
255 / 256	7 / 8

추가 항목	기대
써진 바이트 수	정확히 길이만큼. 모자라도 넘쳐도 FAIL
`WLAST`	각 덩어리의 마지막 박자에만 1
129바이트	목적지 쓰기 2번 (16박자 + 1박자)
512바이트 소스 읽기	4번: `0x1_0000`, `0x1_0080`, `0x1_0100`, `0x1_0180`, 각 16박자
512바이트 목적지 쓰기	4번
512바이트 순서	각 목적지 쓰기 주소는 **대응하는 소스 덩어리의 `RLAST`를 받은 뒤** 나감. 참고 실측(SAN-004): 소스 읽기 주소 3,315 ns, 16박자 뒤 목적지 쓰기 주소 3,495 ns (`RLAST` 시각은 B1로 새로 기록)
1바이트	멈춤·시간 초과 없음

> 실패하면 0-replication 패치 전후 RTL로 같은 시험을 돌려 원인을 나눕니다.

- **흡수한 원래 요구사항**: FUNC-005, FUNC-006, FUNC-007, AXI-002, AXI-003
- **중요도**: 높음

T-08. 중간 알림(EOT)과 descriptor 번호

- **무엇을 확인하나**: 중간 장에 EOT_IRQ 플래그를 켜면 **그 장이 끝날 때도** 알림이 오는지 봅니다. `DESCRIPTOR_ID(0x454)`가 처리 중인 장의 `id`를 순서대로 보여 주는지도 봅니다.
- **왜 중요한가**: T-04가 "체인당 1번"을 본다면, 이 테스트는 "원하면 중간에도"를 봅니다. 둘 다 되어야 알림 제어가 의도대로입니다. `DESCRIPTOR_ID`는 디버깅할 때 "어디까지 갔나"를 보는 창입니다.

- **어떻게 확인하나**:

  1. (중간 알림) 128바이트 descriptor 4장, `id = 0xC0 ~ 0xC3`, `flags = 0b10, 0b00, 0b10, 0b11`로 체인을 만들고 표준 순서로 출발한다.
  2. `irq`가 올라올 때마다: `IRQ_PENDING` 읽기 → 같은 값 쓰기(지우기) → `DESCRIPTOR_ID` 기록 → 횟수 +1. `irq`가 내려갔다가 다시 올라오는지 본다.
  3. (번호 추적) 512바이트 descriptor 8장, `id = 0xD0 ~ 0xD7`, 마지막만 `flags = 0b11`로 체인을 만들고 출발한다.
  4. 전송 중 100클럭마다 `DESCRIPTOR_ID`를 읽어 기록한다 [B6].
  5. 끝난 뒤 `CONTROL ← 0x0`, 이어서 `CONTROL ← 0x1`(SG 모드 끔)을 쓰고 `DESCRIPTOR_ID`를 읽는다.

- **통과 기준** (예외: 1~2단계에서 C4는 "알림이 올 때마다, 지우기 전에 `IRQ_PENDING[1]` = 1"로 봄):

항목	종류	기대
중간 알림 횟수	판정	**3번** (0번, 2번, 3번 장)
1번 장이 끝날 때	판정	알림 없음
번호 추적 순서	판정	`0xD0` → … → `0xD7` 순서. 건너뛰기는 괜찮음, **거꾸로 가면 FAIL**
알림 때마다 읽은 `DESCRIPTOR_ID`	참고 체크, 관측 후 확정	v1.0.0은 `0xC0`, `0xC2`, `0xC3`을 기대함. 하지만 번호는 장이 끝나면 대기열에서 빠지고 그 뒤에 알림이 올라오므로, 다음 장 번호가 보일 수 있음
번호 추적 최종값	참고 체크	`0xD7`
SG 모드를 끈 뒤 `DESCRIPTOR_ID`	참고 체크	`0x0000_0000` (RTL 분석 값)

  > 두 알림이 너무 가까워 하나로 합쳐지면 3번이 2번으로 셀 수 있습니다. 그 실행은 "무효"로 기록하고 다시 돌립니다. 완료는 항상 알림이나 `TRANSFER_DONE`으로 판단합니다. `DESCRIPTOR_ID`로 완료를 판단하지 않습니다.

- **흡수한 원래 요구사항**: DESC-005, DESC-006, REG-016, IRQ-001
- **중요도**: 중간

T-09. 알림 막기(MASK)와 지우기(W1C)

- **무엇을 확인하나**: 막기 설정 4가지에서 알림이 올라오는지, `IRQ_PENDING`과 `IRQ_SOURCE` 값이 각각 무엇인지 봅니다. `IRQ_PENDING`에 1을 쓰면 실제로 `IRQ_SOURCE`가 지워지는지 봅니다.
- **왜 중요한가**: 드라이버가 가장 실수하기 쉬운 곳입니다.
  - 리셋 직후에는 알림이 **다 막혀 있습니다** (`0x3`).
  - 막아 둔 동안에도 일은 `IRQ_SOURCE`에 쌓입니다. 막기를 푸는 순간 옛 알림이 튀어나옵니다.
  - 지우는 곳(`IRQ_PENDING`)과 실제로 지워지는 곳(`IRQ_SOURCE`)이 다릅니다.
- **어떻게 확인하나**:

  1. **HW 리셋을 새로 건 직후**, 다른 쓰기를 하기 전에 `IRQ_MASK`를 읽는다.
  2. 아래 4가지 설정마다:
     - 2.4절 공통 준비(`CONTROL ← 0x0`, `IRQ_PENDING ← 0x3`, 목적지 메모리와 "써졌음" 기록 지우기 [B5])를 한다.
     - `IRQ_MASK`를 설정하고 T-03과 같은 전송을 낸다. 표준 순서의 **2번(`IRQ_MASK ← 0x1`)과 7번(알림 지우기)은 건너뛴다.** 출발 전에 `TRANSFER_ID`를 읽어 둔다.
     - 완료는 **항상 그 번호의 `TRANSFER_DONE` 비트가 1이 될 때까지 반복해서 읽어** 기다린다.
     - `irq` 여부, `IRQ_PENDING`, `IRQ_SOURCE`를 기록한다.

3. 설정 `0x1`에서: 읽은 `IRQ_PENDING` 값(`0x2`)을 그대로 다시 쓴다 → `IRQ_PENDING`, `IRQ_SOURCE`, `irq`를 다시 본다.
4. 2단계를 마친 뒤, 설정 `0x3`으로 전송을 한 번 더 하고 **비우지 않은 채** `IRQ_MASK ← 0x0`으로 막기를 푼다 → `irq`를 본다.
5. 같은 상황을 다시 만들고, 이번엔 `IRQ_PENDING ← 0x3`을 먼저 쓴 뒤 막기를 푼다 → `irq`를 본다.

- **통과 기준** (예외: 이 테스트는 표준 순서 7번을 건너뛰므로, 네 설정 모두 C4를 "`IRQ_SOURCE[1]` = 1이고 그 번호의 `TRANSFER_DONE` 비트 = 1"로 봄):

조건	기대
리셋 직후 `IRQ_MASK`	`0x0000_0003`

`IRQ_MASK`	올라오는 `irq`	`IRQ_PENDING`	`IRQ_SOURCE`
`0x3`	없음	`0x0`	`0x3`
`0x2`	"접수" 알림만	`0x1`	`0x3`
`0x1`	"완료" 알림만	`0x2`	`0x3`
`0x0`	둘 다 (출발 직후 "접수"가 먼저)	`0x3`	`0x3`

지우기 관련	기대
3단계 후 `IRQ_PENDING`	`0x0`
3단계 후 `IRQ_SOURCE`	**`0x1`** — 지운 비트1만 사라지고, 막혀 있던 비트0은 남음
3단계 후 `irq`	내려감
4단계 (안 비우고 막기 해제)	`irq`가 **바로** 올라옴
5단계 (먼저 비우고 막기 해제)	`irq` 안 올라옴

> 비트0 = "접수"(출발 요청이 받아들여짐), 비트1 = "완료".

- **흡수한 원래 요구사항**: IRQ-002, IRQ-003, IRQ-004, IRQ-005
- **중요도**: 높음

T-10. 출발 버튼 규칙과 연속 예약

- **무엇을 확인하나**: `TRANSFER_SUBMIT`(출발 버튼)의 규칙 세 가지와, 체인 A가 도는 동안 체인 B를 미리 예약할 수 있는지 봅니다.
  - 켜기 전에 누르면 조용히 무시된다.
  - 받아들여지면 저절로 0이 된다.
  - 1인 동안 또 눌러도 요청이 늘지 않는다.
- **왜 중요한가**: 켜기 전에 누르면 에러 없이 아무 일도 안 일어나 디버깅이 매우 어렵습니다. 연속 예약이 안 되면 실제 시스템 속도가 떨어지고, 요청이 사라지면 데이터 일부가 안 옮겨집니다.
- **어떻게 확인하나**:
  1. (무시 확인) `CONTROL = 0x0`에서 `TRANSFER_SUBMIT ← 0x1` → 다시 읽고, 버스 활동이 없는지 본다 [B2]. 그다음 체인 두 개를 쓴다:
     - 체인 A: descriptor `0x0000_1300`부터 3장, 소스 `0x0001_A000`부터, 총 1,536바이트, 목적지 `0x0002_8000`
     - 체인 B: descriptor `0x0000_1400`부터 3장, 소스 `0x0001_C000`부터, 총 1,536바이트, 목적지 `0x0002_A000`

    - 각 체인은 마지막 장만 `flags = 0b11`
2. `IRQ_MASK ← 0x1`, `CONTROL ← 0x5`. `TRANSFER_ID`를 읽어 `ID_A`로 적는다. `SG_ADDRESS ← A`, `TRANSFER_SUBMIT ← 0x1` → 0이 될 때까지 반복해서 읽는다.
3. A가 받아들여진 **바로 다음 쓰기로** `SG_ADDRESS ← B`, `TRANSFER_SUBMIT ← 0x1`을 쓰고 **곧바로 다시 읽는다.** (B의 번호 `ID_B`는 `ID_A + 1`이어야 함)
    - 1이면: 한 번 더 `0x1`을 쓴다 (중복 요청 시험).
    - 0이면: B가 이미 받아들여진 것이므로 이번 회차는 "중복 요청 판정 무효"로 적는다. A 체인을 8장 이상으로 늘려 다시 한다. 정해 둔 횟수만큼 해도 안 되면 "판정 불가"로 보고한다.
4. B를 낸 뒤 `ACTIVE_TRANSFER_ID`를 읽고, A가 아직 안 끝났다는 것을 로그로 남긴다 (예약의 증거).
5. 완료는 `TRANSFER_DONE`의 `ID_A`, `ID_B` 비트가 모두 1이 될 때까지 반복해서 읽어 기다린다. 알림이 오면 그때마다 지운다 (지우기 전에 C4 확인). 그다음 `TRANSFER_DONE`, `TRANSFER_ID`, `ACTIVE_TRANSFER_ID`와 두 목적지 데이터를 확인한다.
6. (번호 재사용) 체인 A를 하나씩 3번 더 보낸다. 각 전송은 그 번호의 `TRANSFER_DONE` 비트가 1이 될 때까지 반복해서 읽어 기다리고, 알림을 지운 뒤 다음을 낸다. 번호는 `ID_A + 2`, `ID_A + 3`, 그리고 다시 `ID_A`가 된다. **세 번째를 내기 직전과 직후**에 `TRANSFER_DONE[ID_A]`를 읽는다.

- **통과 기준** (예외: 1단계는 전송이 없는 게 정상이므로 C1~C4 적용 안 함):

항목	기대
1단계 `TRANSFER_SUBMIT` 다시 읽기	`0x0`, 전송 없음, 에러 응답 없음
2·3단계 반복 읽기	끝없이 기다리지 않고 0으로 돌아옴
4단계 `ACTIVE_TRANSFER_ID`	`ID_A` (A가 아직 진행 중)
3단계 중복 요청 (유효한 회차만)	요청이 늘지 않음 — descriptor 읽기 총 횟수 = **A 장수 + B 장수** (기본 3 + 3 = 6번. A를 8장으로 늘린 회차는 8 + 3 = 11번)
5단계 `TRANSFER_DONE`	`ID_A`, `ID_B` 비트 모두 1 — 두 완료를 따로 알아볼 수 있음. 즉 B의 번호는 `ID_A + 1`
5단계 `TRANSFER_ID`	`ID_A + 2` (체인 하나에 번호 하나, 0~3을 돌아감)
5단계 `ACTIVE_TRANSFER_ID`	`TRANSFER_ID`와 같음 (쉬는 상태)
두 목적지 데이터	각각 일치
6단계 `TRANSFER_DONE[ID_A]`	직전 `1` → 직후 `0` (같은 번호로 새로 출발하면 완료 표시가 지워짐)

> `TRANSFER_DONE`은 번호 4개(0~3)만 구분합니다. 끝나지 않은 전송을 4개 넘게 쌓으면 구분이 안 되므로, 이 테스트는 그 전에 확인합니다. 4개를 넘게 쌓는 경우는 이번 15개 범위 밖입니다.

- **흡수한 원래 요구사항**: REG-009, REG-010, REG-011, REG-012, REG-013, REG-014, FUNC-008, IRQ-006
- **중요도**: 높음

T-11. 전송 중 끄기(ENABLE=0)와 다시 켜기

- **무엇을 확인하나**: 체인이 도는 중에 켜기를 끄면, 이미 시작한 버스 거래는 깔끔히 마무리하고 조용해지는지 봅니다. 어떤 값이 지워지고 어떤 값이 남는지, 다시 켜면 새 전송이 정상인지 봅니다.
- **왜 중요한가**: 드라이버가 오류를 고치거나 종료할 때 반드시 쓰는 동작입니다. 덩어리 중간에 끊기면 버스 규칙 위반이고, 조용해지지 않으면 시스템 전체가 멈춥니다.
- **어떻게 확인하나**:

1. 512바이트 descriptor 8장 체인을 공통 영역 안에 만든다 (주소는 TB에서 정해 로그에 남김).
2. 표준 순서로 출발하고, 목적지 쓰기가 몇 번 나간 시점에 `CONTROL ← 0x4`(켜기만 끄고 SG 모드는 유지)를 쓴다.
3. 버스가 조용해질 때까지 모든 요청을 기록한다 [B2]. `DBG_STATUS(0x43C)` 아래 3비트를 반복해서 읽어 기록한다 [B6].
4. 아래 표의 레지스터를 읽는다. 남은 descriptor가 버려지는지 기록한다.
5. `CONTROL ← 0x5`로 다시 켜고 T-03 전송을 보내 정상으로 끝나는지 본다.

- **통과 기준** (예외: 중간에 끊은 체인에는 C2, C4를 적용 안 함. C1은 "조용해질 때까지 기다리는 시간"에 적용):

항목	종류	기대
끈 직후	판정	새 읽기·쓰기 요청이 안 나감
진행 중이던 덩어리	판정	끝까지 마무리 (`WLAST`까지 감)
버스	판정	2.3절 "hang 없음"
`TRANSFER_ID`, `ACTIVE_TRANSFER_ID`, `TRANSFER_DONE`, `TRANSFER_PROGRESS`	판정 (RTL 분석 값)	0
`SG_ADDRESS`, `FLAGS`, `IRQ_MASK`, `IRQ_SOURCE`	판정 (RTL 분석 값)	끄기 전 값 그대로
남은 descriptor 처리	관측 후 확정	—
다시 켠 뒤 전송	판정	정상 완료 (C1~C6 모두)
`DBG_STATUS` 아래 3비트	관찰 기록	SFR 기준 예상: 동작 중 `4`, 조용해진 뒤 `1`. 중간의 `5`(마무리 중)는 짧아서 못 볼 수 있음

> "0 / 그대로" 기대값은 SFR 8장 표에서 왔고, **RTL을 읽고 만든 값이며 아직 측정한 적이 없습니다.** 다르면 RTL 버그인지 표가 틀렸는지 먼저 가립니다.

- **흡수한 원래 요구사항**: REG-006, REG-024 (뒤의 것은 `DBG_STATUS` 관찰 기록으로만 다룸 — 판정 없음)
- **중요도**: 높음

T-12. 일시정지(PAUSE)와 다시 시작

- **무엇을 확인하나**: 전송 중 일시정지를 켜면 멈추고, 끄면 멈춘 자리부터 이어 가는지 봅니다. 데이터가 빠지거나 두 번 써지지 않는지, 진행 바이트 수(`TRANSFER_PROGRESS`)가 실제 진행을 따라가는지 봅니다.
- **왜 중요한가**: 일시정지는 끄기와 달리 상태를 **지우면 안 됩니다.** 다시 시작한 뒤 한 조각이라도 빠지거나 겹치면 데이터가 깨집니다.
- **어떻게 확인하나**:

1. 4 KB descriptor 1장(`x_len = 4095`, 소스 `0x0001_0000`, 목적지 `0x0002_0000`)으로 출발한다.
2. 전송 중 100클럭마다 `TRANSFER_PROGRESS(0x448)`를 읽는다 [B6].
3. **첫 목적지 쓰기 응답(B 응답)이 온 뒤에** `CONTROL ← 0x7`(켜기 + 일시정지 + SG 모드)을 쓴다.
4. 일시정지 뒤에도 이미 나간 요청은 마무리되므로, **세 버스가 조용해질 때까지 기다린다.** 그 뒤 1,000클럭 동안 버스를 보고, `TRANSFER_PROGRESS`를 두 번 이상 읽는다. `DBG_STATUS` 아래 3비트도 기록한다.
5. `CONTROL ← 0x5`로 일시정지를 푼다. 끝날 때까지 기다린 뒤 데이터를 비교하고, `TRANSFER_PROGRESS`를 읽는다.

- **통과 기준**:

항목	종류	기대
조용해진 뒤 1,000클럭	판정	새 요청 없음
그때 `TRANSFER_PROGRESS`	판정	0보다 크고 4,096보다 작으며, 읽을 때마다 같음
일시정지 중 `TRANSFER_ID` 등 추적 값	판정 (RTL 분석 값)	지워지지 않음
다시 시작 후	판정	멈춘 자리부터 이어 감
데이터	판정	4,096바이트 모두 일치, 빠짐·겹침 없음
전송 중 `TRANSFER_PROGRESS`	판정	0부터 늘어나고 4,096을 넘지 않음
끝난 뒤 `TRANSFER_PROGRESS`	판정 (RTL 분석 값)	0 (끝난 다음 클럭에 지워짐)
일시정지 중 `DBG_STATUS` 아래 3비트	관찰 기록	SFR 기준 예상 `2`
체인에서 `TRANSFER_PROGRESS`가 장을 넘어 계속 쌓이는지	관측 후 확정	512바이트 2장 체인으로 따로 관측

- **흡수한 원래 요구사항**: REG-008, REG-018
- **중요도**: 중간

T-13. 주소 정렬과 4 KB 경계

- **무엇을 확인하나**: 8바이트에 맞지 않는 주소를 주면 어떻게 되는지 봅니다. 데이터나 descriptor가 4 KB 주소 경계에 걸칠 때 AXI 규칙("한 덩어리는 4 KB 경계를 넘으면 안 됨")을 지키는지 봅니다.
- **왜 중요한가**: descriptor는 48바이트를 **한 덩어리로** 읽기 때문에, 경계에 걸치면 규칙 위반이 될 수 있습니다. 이건 SG에만 있는 실패입니다. 정렬 안 된 주소의 동작은 아직 아무도 정하지 않았습니다.
- **어떻게 확인하나**:
  1. (descriptor 주소 아래 3비트) 정상 descriptor 주소에 3을 더한 값을 `SG_ADDRESS`에, 또 앞 장의 "다음 장 주소"에 넣어 본다. 실제 descriptor 읽기 주소를 기록한다 [B2].
  2. (데이터 경계) 소스 `0x0000_0F80`, 길이 256바이트로 보낸다. 이 전송의 descriptor는 BOUND 영역 밖(`0x0000_1100` 이상, 64바이트 정렬)에 둔다.
  3. (정렬 안 됨 + 경계) 소스 `0x0000_0F83`, 길이 256바이트로 보낸다.
  4. (descriptor 경계) descriptor를 `0x0000_0FF0`에 둔다. 48바이트가 `0x1000` 경계에 걸친다. 이 단계는 B1의 descriptor 모양 감시와 4 KB 감시를 기록만 하도록 바꾼다.
  5. (정렬 안 된 주소 모음) 길이 64바이트로 (소스 / 목적지 어긋남) = (+0/+0), (+1/+0), (+0/+1), (+1/+1), (+3/+5), (+7/+1)을 돌린다. 반복마다 메모리를 지운다 [B5].
- **통과 기준** (예외: 3·4·5단계는 C2를 관측 후 확정, 단 5단계 (+0/+0)는 판정. 4단계는 C5를 기록만 함):

항목	종류	기대
1단계 descriptor 읽기 주소	판정	아래 3비트가 0
2·3단계 모든 데이터 덩어리	판정	4 KB 경계를 넘지 않음. **넘으면 FAIL**
2단계 데이터	판정	C2, C3 그대로
3단계 데이터	관측 후 확정	—
4단계 descriptor 읽기	관측 후 확정	DMA가 나눠 읽는지, 경계를 넘는지 기록. 넘으면 "descriptor는 64바이트 정렬 필수"를 SW 제약으로 올림 (SFR은 64바이트 정렬을 권장)

항목	종류	기대
5단계	관측 후 확정	첫 실행은 판정하지 않고 관측만. 결과를 보고 기대값을 정한 뒤 회귀 시험으로 바꿈
5단계 (+0/+0)	판정	C1~C3 통과

- **흡수한 원래 요구사항**: AXI-004, DESC-008, FUNC-010
- **중요도**: 중간

T-14. 느린 버스(backpressure)와 AXI 속성

- **무엇을 확인하나**: 메모리 쪽이 `READY`를 늦게 줘도(느리게 받아도) 데이터가 정확한지 봅니다. 세 포트가 내보내는 `AxCACHE`, `AxPROT` 값이 설정과 같은지 봅니다.
- **왜 중요한가**: 지금 TB 메모리는 지연 없이 바로 응답하지만 실제 칩은 그렇지 않습니다. 특히 **descriptor 읽기만 느릴 때** 데이터 전송이 제대로 기다리는지는 SG만의 스트레스 조건입니다. 레지스터 값(SAN-001)이 맞아도 실제 버스 출력은 따로 봐야 합니다.
- **어떻게 확인하나**:

  1. 느린 응답 주입 [B4]과 AXI 규칙 검사기 [B1]을 켠다.
  2. T-04의 3장 gather 체인을 아래 5가지 지연 조합으로 반복한다. 반복마다 메모리를 지운다 [B5].
  3. `VALID`를 올린 뒤 `READY`가 올 때까지 주소·데이터가 안 바뀌는지 자동 검사한다.
  4. 모든 거래에서 세 포트의 `AxCACHE`, `AxPROT`를 검사한다.

반복	descriptor 포트 지연 (클럭)	소스 포트 지연	목적지 포트 지연
1	0	0	0~15
2	0	0~15	0
3	**0~15**	0	0
4	0~7	0~7	0~7
5	0~15	0~15	0~15

- **통과 기준**:

항목	기대
5개 조합 데이터	모두 100% 일치
지연의 영향	끝나는 시간만 늘고 결과는 같음
`VALID` ~ `READY` 사이	주소·데이터 유지 (위반 0)
모든 포트 `AxCACHE`	`4'b0000`
모든 포트 `AxPROT`	`3'b000`

- **흡수한 원래 요구사항**: AXI-005, AXI-007
- **중요도**: 중간

T-15. 에러 응답과 잘못된 descriptor

- **무엇을 확인하나**: 메모리가 에러 응답을 주거나 descriptor 내용이 극단적일 때 DMA가 어떻게 행동하는지 봅니다. 최소한 **hang이 없는지** 봅니다.
- **왜 중요한가**: 이 부분은 ADI testbench도 우리 TB도 한 번도 건드리지 않았습니다. RTL을 보면 descriptor를 읽을 때 에러 응답을 **검사하지 않습니다.** 그러면 잘못된 descriptor를 읽어도 에러 표시 없이 엉뚱한 주소로 전송할 수 있고, 시스템 메모리를 망가뜨릴 수 있습니다.
- ⚠️ **이 테스트는 동작이 아직 정해지지 않았습니다.** 먼저 RTL(`response_handler.v`, `axi_dmac_response_manager.v`, `src_axi_mm.v`, `dest_axi_mm.v`)에서 응답 처리를 읽고, 시뮬레이션으로 관측해 동작을 정한 뒤 자세한 기대값을 정합니다.
- **어떻게 확인하나**: 각 경우마다 정해 둔 시간을 기다린 뒤 2.3절 "hang 없음" 절차(끄기 → 조용해짐 확인 → 다시 켜기 → T-03 전송)를 한다.
  1. (소스 읽기 에러) 소스를 ERR 영역에 두고 읽기 응답을 SLVERR로 바꾼다 [B3]. DECERR로 한 번 더 한다. 목적지를 ERR 영역에 두고 쓰기 응답을 SLVERR로 바꾸는 것도 한다.
  2. (descriptor 읽기 에러) 3장 체인 중 **두 번째 장만** ERR 영역에 두고 읽기 응답을 SLVERR로 바꾼다 [B3]. 이때 돌려주는 데이터는 모두 0이다. 에러를 검사하지 않으므로 이 0이 그대로 descriptor로 쓰일 수 있다 (`flags = 0`, 다음 장 주소 0 → 주소 0을 계속 읽을 수 있음).
  3. (최대 길이) `x_len = 0xFFFFFF` (**16 MiB**)인 descriptor로 출발한다. 목적지는 다른 영역과 겹치지 않는 곳으로 TB에서 정한다. **첫 목적지 쓰기를 확인한 뒤** 끈다. 끝까지 보내는 시험은 시뮬레이션 시간 문제로 별도 검토로 미룬다.
  4. (자기 자신을 가리키는 체인) `flags = 0b00` (마지막 아님, 알림 없음)이고 "다음 장 주소"가 자기 주소인 descriptor로 출발한다.
  5. 각 경우마다 `irq`, `IRQ_SOURCE`, `TRANSFER_DONE`, 버스 활동, 목적지 메모리를 기록한다.
- **통과 기준** (예외: C1~C4 적용 안 함. 이 테스트 동안 B1 검사기는 **기록만** 하도록 바꾸고(`$error` 대신 `$info`), C5는 관찰 기록. C6은 B1 이외의 `$error`에만 판정):

경우	판정	자세한 동작 (관측 후 확정)
소스 읽기 에러 (SLVERR, DECERR)	2.3절 "hang 없음"	에러 표시가 있는지, 목적지에 무엇이 써졌는지
목적지 쓰기 에러	위와 같음	에러 표시가 있는지, 완료 기록이 남는지
descriptor 읽기 에러	위와 같음	에러 표시 없이 진행할 가능성이 높음. 그렇다면 "DMA가 알아챌 수 없음 → 칩 수준 보호 필요"를 제약으로 기록
16 MiB 전송	위와 같음	원래 시간 안에 안 끝나는 것이 정상. 써진 범위가 지정 범위 안인지
자기 자신을 가리키는 체인	위와 같음	SFR상 정상 구성. 끄기 전까지 계속 도는 것이 정상

> **ERR-004에 대한 결론**: v1.0.0이 말한 "길이 0 descriptor"는 이 형식으로 **만들 수 없습니다** (`x_len = 0`은 1바이트). 그래서 이 테스트는 그 자리에서 최대 길이(16 MiB)와 자기 참조 체인을 봅니다. hang이 나면 그 자체가 중요한 발견입니다. 재현 조건과 파형을 남깁니다.

- **흡수한 원래 요구사항**: ERR-001, ERR-002, ERR-003, ERR-004
- **중요도**: 높음

5. 제외한 요구사항

원래 ID	내용	제외 이유
REG-017	`CURRENT_SRC_ADDRESS(0x438)` / `CURRENT_DEST_ADDRESS(0x434)`가 진행에 따라 바뀜	디버그 레지스터라 값 비교를 하지 않는 규칙 대상이고, 값이 버스 타이밍에 따라 달라 정답을 정할 수 없음. 진행 상황은 T-12의 `TRANSFER_PROGRESS`로 봄
AXI-006	응답이 안 온 요청이 여러 개 있어도 정상 동작	지금 TB 메모리는 요청을 한 번에 하나만 받아서 시험 자체가 불가능. 메모리 모델을 바꾼 뒤 성능 측정과 함께 다룸
COV-001	가능하면 toggle coverage 측정	테스트가 아니라 측정 활동. 15개 테스트를 마친 뒤 따로 함

> REG-024(`DBG_STATUS`)는 T-11에 흡수했지만, 디버그 레지스터라 **관찰 기록으로만** 다룹니다. 즉 58건 중 REG-024 한 건은 판정 없이 기록만 합니다.

v1.0.0 시나리오 중 넣지 않은 것

시나리오	내용	처리
ET-004	전송 중 리셋	연결된 요구사항이 없어 넣지 않음
CT-005	최대 길이(2^24 − 1) 근처	T-15의 16 MiB 시험이 일부 대신함. 끝까지 보내는 시험은 시뮬레이션 시간 때문에 별도 검토
CT-007	소스와 목적지가 겹침	동작 파악용 시나리오. 연결 요구사항(FUNC-001)은 T-03이 판정
RT-001, RT-002	무작위 체인, 무작위 + 지연	무작위 시험은 15개 범위 밖. 지연은 T-14가 고정 조합으로 대신함

---

6. 원래 61건 대응표

6.1 ID별 배정

원래 ID	테스트	원래 ID	테스트	원래 ID	테스트
REG-001	T-01	REG-022	T-02	DESC-005	T-08
REG-002	T-01	REG-023	T-02	DESC-006	T-08
REG-003	T-01	REG-024	T-11	DESC-007	T-03
REG-004	T-01	IRQ-001	T-08	DESC-008	T-13
REG-005	T-01	IRQ-002	T-09	DESC-009	T-04
REG-006	T-11	IRQ-003	T-09	FUNC-001	T-03
REG-007	T-03	IRQ-004	T-09	FUNC-002	T-04
REG-008	T-12	IRQ-005	T-09	FUNC-003	T-05
REG-009	T-10	IRQ-006	T-10	FUNC-004	T-06
REG-010	T-10	AXI-001	T-03	FUNC-005	T-07
REG-011	T-10	AXI-002	T-07	FUNC-006	T-07
REG-012	T-10	AXI-003	T-07	FUNC-007	T-07
REG-013	T-10	AXI-004	T-13	FUNC-008	T-10

원래 ID	테스트	원래 ID	테스트	원래 ID	테스트
REG-014	T-10	AXI-005	T-14	FUNC-009	T-03
REG-015	T-03	AXI-006	제외	FUNC-010	T-13
REG-016	T-08	AXI-007	T-14	ERR-001	T-15
REG-017	제외	DESC-001	T-03	ERR-002	T-15
REG-018	T-12	DESC-002	T-03	ERR-003	T-15
REG-019	T-02	DESC-003	T-03	ERR-004	T-15
REG-020	T-01	DESC-004	T-04	COV-001	제외
REG-021	T-02				

6.2 집계

영역	원래 건수	흡수	제외
REG	24	23	1 (REG-017)
IRQ	6	6	0
AXI	7	6	1 (AXI-006)
DESC	9	9	0
FUNC	10	10	0
ERR	4	4	0
COV	1	0	1 (COV-001)
합계	**61**	**58**	**3**

- 61건이 모두 딱 한 번씩 나옵니다 (스크립트로 확인).
- SG 고유 요구사항 18건(REG-007, REG-015, REG-016, AXI-001, DESC-001~009, FUNC-002~004, ERR-002, ERR-004)은 **모두 흡수**했습니다.

6.3 v1.0.0 시나리오와의 대응

테스트	v1.0.0 시나리오
T-01	SAN-001, SAN-002
T-02	SAN-007, FT-014 (1~3번)
T-03	SAN-004
T-04	SAN-005
T-05	FT-001
T-06	FT-002, CT-004
T-07	FT-003, FT-005
T-08	FT-006, FT-008
T-09	SAN-003, SAN-006, FT-013
T-10	FT-007, FT-014 (4번), CT-006 (20회 연속 대신 번호 재사용 4회까지만)
T-11	FT-011, FT-010 (`DBG_STATUS` 부분)
T-12	FT-012, FT-010 (일부)

테스트	v1.0.0 시나리오
T-13	CT-001 ~ CT-003, FT-004
T-14	FT-009
T-15	ET-001 ~ ET-003, ET-005, CT-008

---

7. 개정 이력

판	날짜	내용
rel-v1.0.0	2026-10	요구사항 61건
rel-v2.0.0	2026-10-16	61건을 테스트 15개로 줄임 (흡수 58, 제외 3). 처음 보는 사람도 읽을 수 있게 다시 씀. 고친 점: "길이 0 descriptor" 오류 수정, "hang 없음" 정의, 디버그 레지스터는 판정에 안 씀, 끝난 뒤 `DESCRIPTOR_ID`는 참고 체크, testbench 보강 목록 추가. 근거: `SGDMA-SOR-INTEGRATION-v1.0`